Systems Architecture

Sequential logic or *Flip-Flops*

Sequential Logic

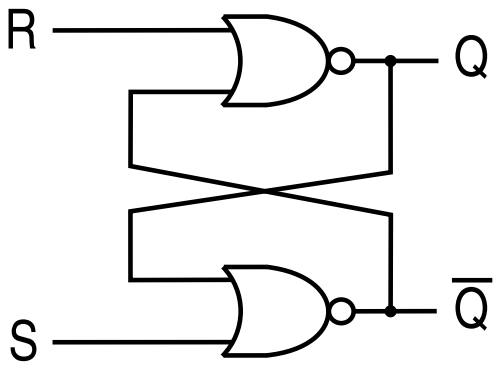
- Remembers *past* inputs
- Output is stable (always 1 or 0)
- *flip* into one state, *flop* into the other
- Normally *clocked*

The R-S Latch

R	S	Q
0	0	*Q*
0	1	1
1	0	0
1	1	–

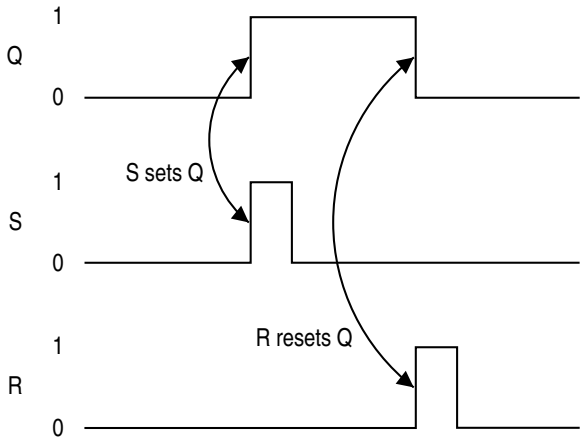
S = Set

R = Reset



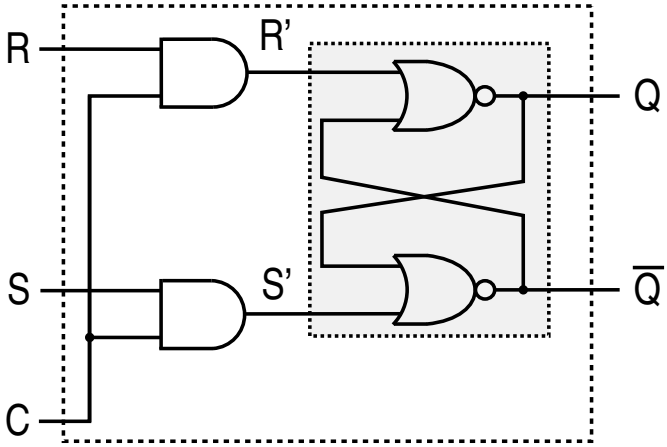
Use of the RS Latch

Capture short-lived events



The clocked RS Flip-Flop

Pays attention to R and S only when *clocked*



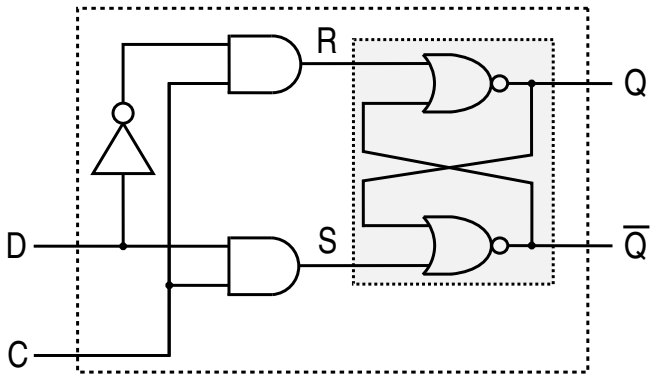
The D Flip-Flop

When clocked, capture the value of D

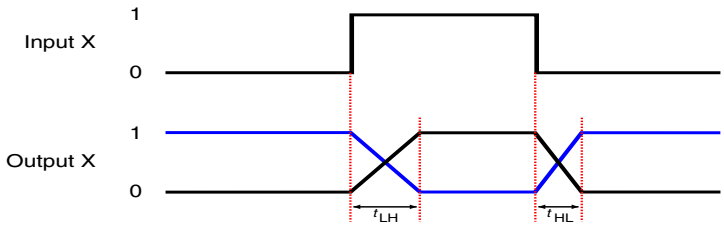
C	D	Q
0	0	*Q*
0	1	*Q*
1	0	0
1	1	1

C = Clock

D = Data

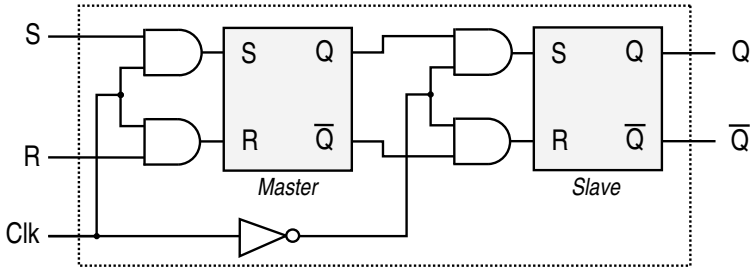


Clocking



- Clock gives circuit the “Go” signal
- Three types of clock
  level-sensitive (high or low)
  edge-sensitive (rising or falling)
  master-slave
- Timing problems (clock skew)

The Master-Slave Flip-Flop



Master - slave RS Flip-Flop

- Two RS flip-flops in series
- First (*master*) clocked on rising edge
- Second (*slave*) clocked on falling edge
- Master protects slave from clock skew

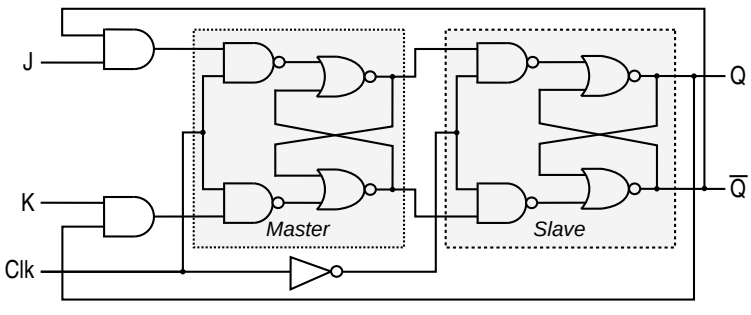
The J-K Flip-Flop

RS Flip-Flop with toggle on J = K = 1

J	K	Q
0	0	Q
0	1	0
1	0	1
1	1	$\overline{Q}$

J = Set

K = Reset



Summary of Flip-Flops

- Have an *internal state* or *memory*
- Use a *clock* to trigger the flip-flop
- Three common types:

D Captures D when clocked

RS Programmed: Reset or Set when clocked
Reset and Set both off (0) is ignored
Reset and Set both on (1) is forbidden

JK An RS flip-flop with toggle, when clocked will:
Reset (K) or Set (J) the output
Reset and Set both off (J=K=0) is ignored
Reset and Set both on (J=K=1) output is inverted